

SECTION 2 DATA STRUCTURES USING PYTHON LAB

Structure

2.0 Introduction

2.1 Objectives

2.2 Programming Exercises for Lab Sessions

2.3 Summary

2.4 Further Readings

2.0 INTRODUCTION

Data Structures is one of the most foundational subjects in computer science. Data structures define how data is stored, organized, and managed in memory. Choosing the right structure (array, linked list, tree, hash table, etc.) directly impacts how efficiently a program can access and manipulate data. Almost every algorithm relies on an underlying data structure. The choice of data structure determines the time and space complexity of operations. For example, searching in a hash table is $O(1)$ on average, while a linear search in an array is $O(n)$. Most real-world programming problems, from routing in networks to managing databases, require a solid understanding of data structures. They give you the mental toolkit to break down and solve complex problems systematically. Good software design depends on selecting appropriate data structures. Whether you're building a file system, a search engine, or a social network, the internal organization of data shapes the entire architecture. Data structures are prerequisites for many advanced areas, including operating systems, compilers, databases, machine learning, and distributed systems. Data structures (along with algorithms) form the core of technical interviews at top tech companies. Mastery signals strong problem-solving ability and maturity. Understanding structures like stacks and heaps helps programmers write memory-efficient code and debug issues like memory leaks or stack overflows. In short, Data Structures teaches you not just about writing code that works, but code that works efficiently.

Python is one of the most important and widely used programming languages in the world today. Python's syntax is clean, intuitive, and close to plain English. This makes it beginner-friendly while remaining powerful enough for expert-level work. You spend less time debugging syntax and more time solving problems. Python is used across a remarkably wide range of fields, namely, web development (Django, Flask, FastAPI), Data Science (Pandas, NumPy, Matplotlib), Machine Learning & AI (TensorFlow, PyTorch, Scikit-learn), Automation & Scripting (File handling, web scraping), Cybersecurity (Penetration testing, threat analysis), Game Development (Pygame), Embedded Systems (MicroPython, Raspberry Pi). Python is the de facto language of Artificial Intelligence, Machine Learning, and Data Science. If you want to work in these fields, Python is non-negotiable. Python has a very large number of packages on PyPI (Python Package Index). Whatever you want to build, there's likely a library for it, saving enormous development time. Python consistently ranks among the most in-demand programming languages globally. Many reputed companies heavily use Python. Python has one of the largest and most active open-source communities in the world. It means abundant tutorials and documentation and constantly evolving libraries and tools. Python allows developers to build and test ideas quickly. Python enables programmers

to write short programs for some functionalities which otherwise require long programs in other programming languages thus accelerating development cycles. Python runs seamlessly on Windows, macOS, and Linux usually without modification, making it highly portable. Studying Python opens doors to Cloud computing (AWS Lambda, Google Cloud Functions), DevOps & automation (Ansible, Fabric), Big Data (Apache Spark with PySpark), and APIs and microservices. Also, tools like Jupyter Notebook make it ideal for academic work.

Studying Data Structures using Python offers many advantages. Python's clean and readable syntax lets you focus on understanding the logic of data structures rather than getting lost in complex syntax rules. Python comes with powerful builtin structures out of the box, making it easy to learn concepts quickly and then work deeper. Some of the Python's built in data structures are *list*, *tuple*, *dict*, *set*, *collections.deque*, *heapq* corresponding to the concepts of array/dynamic array/stack, immutable array, hash table/dictionary, hash set, double ended queue, priority queue/heap. These built in data structures can be used first by understanding their behavior, and then implement them from scratch for deeper understanding. You can use them first, understand their behavior, and then implement them from scratch for deeper learning. Python allows to implement complex data structures in fewer lines of code, saving time and reducing errors. This speed lets us experiment more and learn faster. Python's dynamic typing means data structures can hold any type of data without strict type. This flexibility helps beginners focus on the structure and logic rather than type management. Python offers powerful libraries that complement learning data structures. The purpose of *collections* library is to offer advanced data structures such as deque, OrderedDict, Counter. The purpose of *heapq* library is to offer support for heap operations. The purpose of *array* library is to offer support for typed arrays. The purpose of *sortedcontainers* library is to offer support for sorted lists, dicts, sets. The purpose of *networkx* library is to offer support for graph data structures. The purpose of *matplotlib* library is to offer support for visualizing data structures. With libraries like *Matplotlib* and *NetworkX*, it is possible to visually render data structures making abstract concepts concrete. Seeing a tree or graph visually leads to increase in understanding. Python's *Jupyter Notebook* environment lets us to write and run code cell by cell, view output immediately, add notes and explanations alongside code and experiment without restarting the whole program. This is ideal for learning data structures step by step. Data structures like Trees, Graphs, and Linked Lists are naturally modeled using classes and objects and Python's support for OOP will make it easy to implement. Python is used in industry for AI, web development, and data science and all of them rely heavily on data structures. Studying data structures using or in Python means that knowledge can be immediately applicable to real projects. Python has lot of programming communities in the world. It means large number of tutorials on data structures in Python; active forums such as Stack Overflow, Reddit, GitHub; many courses on Coursera, edX, YouTube; then popular books like *Data Structures and Algorithms in Python* by Goodrich et al. Most coding interview platforms such as LeetCode, HackerRank, and Codeforces support Python. So, studying data structures in Python means you're directly preparing for technical interviews. Many data structure operations such as tree traversal, and graph search use recursion and Python supports recursion. Studying Data Structures using Python enables us to learn the concepts deeply without being burdened by lowlevel complexities. It's one of the best ways to build a strong computer science foundation. At the same time, we are simultaneously gaining one of the most in-demand industry skills.

2.1 OBJECTIVES

After studying this section and implementing the lab exercises, you should be able to

- learn the importance of Data Structures;
- learn the importance of Python;
- experience the advantages of learning Data Structures using Python; and
- acquire skills to implement the Data Structures that are not covered in lab exercises.

2.2 LIST OF RESOURCES

The following is a list of some resources to learn Python / Data Structures using Python as on date:

1. NOC: The Joy of Computing using Python (NPTEL)
<https://nptel.ac.in/courses/106106182>
2. NOC: Programming, Data Structures and Algorithms using Python (NPTEL)
<https://nptel.ac.in/courses/106106145>
3. Python Tutorial
<https://www.w3schools.com/python/default.asp>
4. Python Tutorial
<https://www.geeksforgeeks.org/python/python-programming-language-tutorial/>
5. Python Data Structures
<https://www.coursera.org/learn/python-data>
6. Data Structures and Algorithms with Python
<https://www.geeksforgeeks.org/dsa/python-data-structures-and-algorithms/>
7. Data Structures and Algorithms with Python
https://www.w3schools.com/python/python_dsa.asp

2.3 LAB SESSIONS

There are a total of 20 lab sessions in MCSL-064 (Data Structures using Python Lab). Each lab session is of 3 hours duration. So, in total, you have 60 hours of lab. In this section, you will find exercises for all the lab sessions, session-wise. However, please note that, from learner to learner time may vary for completion of each lab exercise. So, the objective is that, together in 20 lab sessions of 60 hours in total, you should be able to complete all the given lab exercises. A total of 39 lab exercises were prescribed for 20 lab sessions. It is strongly suggested that, for each lab exercise, you write algorithm, and program before entering the lab. After you write the program, hand simulate it on paper. It will enable you to update the code before entering the lab. These steps will ensure that you optimally utilize the lab time for executing the programs. In lab, you are expected to utilize the machine time for executing the programs successfully. Don't use any AI tools to generate the code.

The following are the lab sessions along with the corresponding lab exercises:

Session-1 (3 hours) : Python Basics

1. Write a program in Python to count the number of words, blanks, characters in input string.
2. Write a program in Python to replace multiple consecutive blanks in a string with a single blank.

Session-2 (3 hours) : Python Basics

3. Write a program in Python to reverse a string using slicing and functions.
4. Write a program in Python to convert lowercase characters to uppercase and vice versa in a string.

Session-3 (3 hours) : Recursion

5. Write a program in Python to compute the frequency of each word in a given text using a dictionary.
6. Write a recursive Python function to compute the factorial of a number.

Session-4 (3 hours) : Recursion

7. Write a program in Python to generate the Fibonacci sequence up to n terms (both iterative and recursive versions).
8. Write a recursive Python function to reverse a string.

Session-5 (3 hours) : Recursion

9. Write a recursive function to compute the product of two integers using only addition.
10. Write a recursive Python function to generate all subsets of a given set.

Session-6 (3 hours) : Stacks

11. Write a program in Python to implement a Stack using Python lists and perform push, pop, and peek operations.
12. Write a program in Python to evaluate a postfix expression using a stack.

Session-7 (3 hours) : Queues & Deque

13. Write a program in Python to implement a Queue using Python lists and perform enqueue and dequeue operations.
14. Write a program in Python to implement a Deque (double-ended queue).

Session-8 (3 hours) : Stacks & Queues

15. Write a program in Python to implement a Queue using two stacks.

Session-9 (3 hours) : Linked Lists

16. Write a program in Python to implement a Singly Linked List with insertion, deletion, and traversal operations.
17. Write a program in Python to implement a Doubly Linked List with insertion and deletion operations.

Session-10 (3 hours) : Trees

18. Write a program in Python to implement a Binary Tree. The program should also provide for insertion operation.

19. Write recursive functions in Python for preorder, inorder, and postorder traversal of a Binary Tree.

Session-11 (3 hours) :Trees

20. Write a program in Python that includes non-recursive preorder, inorder and postorder traversals of a Binary Tree.

Session-12 (3 hours) :Trees

21. Write a program in Python to implement a Binary Search Tree (BST) with insertion, deletion, and search operations.

Session-13 (3 hours) :Searching & Sorting

22. Write a program in Python to implement Binary Search. Implement both iterative and recursive versions.

23. Write a program in Python to implement Selection Sort.

Session-14 (3 hours) :Sorting

24. Write a program in Python to implement Insertion Sort.

25. Write a program in Python to implement Merge Sort using divide-and-conquer strategy.

Session-15 (3 hours) :Sorting

26. Write a program in Python to implement Quick Sort. Also, compare its performance with Merge Sort.

27. Write a program in Python to implement Heap Sort.

Session-16 (3 hours) :Applications

31. Write a program in Python to add two polynomials.

32. Write a program in Python for addition of two sparse matrices.

Session-16 (3 hours) :Applications

33. Write a program in Python to perform multiplication of two sparse matrices.

Session-17(3 hours) :Graphs

34. Write a program in Python to implement Dijkstra's algorithm to find the shortest path from a source vertex.

35. Write a program in Python to implement Kruskal's algorithm to find the Minimum Cost Spanning Tree of a graph.

Session-18 (3 hours) : Graphs

36. Write a program in Python to implement Topological Sort for a directed acyclic graph (DAG).

37. Write a program in Python to implement Prim's Algorithm for finding the Minimum Cost Spanning Tree.

Session-19 (3 hours) : Pattern Matching Algorithms

38. Write a program in Python to implement Knuth-Morris-Pratt (KMP) string matching algorithm.

Session-20 (3 hours) : Pattern Matching Algorithms

39. Write a program in Python to implement Boyer-Moore string matching algorithm.

2.3 SUMMARY

In this section, you were introduced to the importance of data structures, importance of Python programming language and most importantly about the advantages of learning data structures using Python. A total of 39 lab exercises for 20 lab sessions (60 hours of lab) were given and completion of all the lab exercises will enable you to acquire skills for implementation of Data Structures that are not part of the lab exercises.

2.4 FURTHER READINGS

Online Resources

1. freeCodeCamp — Learn Python Full Course for Beginners
<https://www.youtube.com/watch?v=rfscVS0vtbw>
2. freeCodeCamp — Ultimate Beginner's Python Course
<https://www.youtube.com/watch?v=qwAFL1597eM>
3. Corey Schafer — Python Tutorials Playlist
<https://www.youtube.com/playlist?list=PL-osiE80TeTt2d9bfVyTiXJA-UTHn6WwU>
4. Programming with Mosh - Python Tutorials

https://youtube.com/playlist?list=PLTjRvDozrdlxj5wgH4qkvwSOdHLOCx10f&si=GVilpsV_fTASnHvK

5. Telusko — Python Tutorial for Beginners

<https://www.youtube.com/playlist?list=PLsyebzWxl7poL9JTVyndKe62ieoN-MZ3>

6. Tech With Tim — Python Tutorials

<https://www.youtube.com/playlist?list=PLzMbGfZo4-mFu00qxl0a67RhjjZj3jXm>

7. Python Essentials 1 (Self-Paced Online)

<https://www.netacad.com/courses/python-essentials-1?courseLang=en-US>

8. Python Essentials 1 (Self-Paced Online)

<https://www.netacad.com/courses/python-essentials-2?courseLang=en-US>

Reference Books

- Python Crash Course: A Hands-On, Project-Based Introduction to Programming by Eric Matthes; No Starch Press.
- Data Structures and Algorithms in Python by Michael T. Goodrich, et al.; Wiley

THE PEOPLE'S
UNIVERSITY